

INFORMAÇÕES SOBRE A AVALIAÇÃO

AV1 AVALIAÇÃO 1 - 25 pontos

INFORMAÇÕES DOCENTE

CURSO:	DISCIPLINA:	TURNO	MANHÃ	TARDE	NOITE	PERÍODO/SALA:
ENGENHARIA DE SOFTWARE	DIAW		x	x	x	2º
PROFESSOR (A): João Paulo Carneiro Aramuni						

INFORMAÇÕES DISCENTE

ALUNO(A):	RA:
DATA:	NOTA:

INSTRUÇÕES:

- Consulta:
 - Será permitido consultar os seus próprios projetos desenvolvidos em laboratório.
 - Será permitido consultar a documentação oficial do Spring Boot (spring.io) e do W3C (w3.org) para HTML e CSS.
 - Não será permitido o uso de inteligência artificial, como o ChatGPT, Claude AI, Gemini, GitHub Copilot, etc., bem como consultas ao Google ou a qualquer outro site de busca.
- Tecnologias:
Será obrigatório o uso de Java 25, Spring Boot, Apache Maven, Thymeleaf, HTML e CSS.
- Aparelhos eletrônicos:
Não será permitido o uso de celulares, relógios digitais ou qualquer outro aparelho eletrônico durante a realização da prova.
- Conduta:
O aluno que, durante a prova, utilizar meios ilícitos, como comunicação com outros alunos, consulta a qualquer tipo de material extra prova ou uso de aparelhos eletrônicos, será eliminado do processo avaliativo e receberá nota 0 (zero).
- Funcionamento:
Projetos que não rodam, ou seja, que não compilam, ou que não possam ser acessados via <http://localhost:8080/> serão avaliados com até no máximo 50% da nota da prova (12,5 pts).

Boa Prova!

Sistema de Consulta de Candidatos TSE

[10,0 pts] Questão 1) Construa o arquivo **controller/CandidatosTseController.java** e realize as chamadas necessárias para os arquivos disponibilizados na prova, chamados **service/CandidatosTseService.java** e **model/Candidato.java**.

Dicas:

- O Controller é uma classe anotada com `@Controller` (não `@RestController`, pois ele retorna uma view do Thymeleaf, não JSON).
- Injete o `CandidatosTseService` via construtor:

```
@Controller
public class CandidatosTseController {

    private final CandidatosTseService candidatosTseService;

    public CandidatosTseController(CandidatosTseService candidatosTseService) {
        this.candidatosTseService = candidatosTseService;
    }
    // ...
}
```

- Crie um único endpoint (GET /) que atenda tanto o carregamento inicial da página quanto os filtros; os filtros são simplesmente parâmetros de query string (?cargo=...&partido=...&texto=...).
- Use @RequestParam(required = false) para cada filtro, já que o usuário pode não preencher nenhum:

```
@GetMapping("/")
public String index(
    @RequestParam(required = false) String cargo,
    @RequestParam(required = false) String partido,
    @RequestParam(required = false) String texto,
    Model model) {
    // ...
}
```

- Confira a assinatura exata do método filtrar(...) no CandidatosTseService fornecido; os parâmetros e a ordem precisam bater certinho.
- Você vai precisar passar pro Model (para o Thymeleaf usar no HTML):
 - a lista filtrada de candidatos;
 - o total de candidatos encontrados (útil pra mensagem "X candidato(s) encontrado(s)" e pra saber quando mostrar "nenhum resultado");
 - as opções de filtro (lista de cargos e de partidos distintos), o service já deve ter métodos prontos para isso (algo como listarCargos() e listarPartidos());
 - os valores atualmente selecionados de cada filtro, para que o formulário "lembre" o que o usuário digitou/selecionou depois de recarregar a página.
- Cuidado com null: se o parâmetro vier null, ao devolver pro HTML prefira transformar em string vazia (""), assim evita null aparecendo escrito na tela ou quebrando comparações no Thymeleaf.
- O método deve retornar o nome da view (uma String), não o objeto HTML, o Thymeleaf resolve isso a partir do nome (ex.: return "index"; aponta pra templates/index.html).

[10,0 pts] Questão 2) Construa o arquivo templates/index.html

Dicas:

Estrutura geral da página

- Declare o namespace do Thymeleaf no <html>:
 - <html lang="pt-BR" xmlns:th="http://www.thymeleaf.org">
- Referencie o CSS com th:href (não href puro), para que o Thymeleaf resolva o caminho junto com o contexto da aplicação:
 - <link rel="stylesheet" th:href="@{/css/style.css}">

O formulário de filtro

- Use `method="get"` — assim os filtros viram parâmetros de URL, sem precisar de JavaScript nem de outro endpoint:
 - `<form method="get" th:action="@{/}" class="filtros">`
- Para o campo de texto, use `th:value` pra manter o que o usuário digitou depois de filtrar:
 - `<input type="text" name="texto" th:value="{textoSelecionado}">`
- Para os `<select>` de cargo/partido, você vai precisar de duas coisas: uma opção "Todos" (valor vazio) e as opções dinâmicas vindas da lista do Model, usando `th:each`. Preste atenção em marcar qual opção deve ficar selecionada por padrão, comparando o valor da opção com o valor atualmente selecionado:

```
<select name="cargo">
  <option value="" th:selected="{cargoSelecionado == ''}">Todos</option>
  <option th:each="c : {cargos}"
    th:value="{c}"
    th:text="{c}"
    th:selected="{c == cargoSelecionado}"></option>
</select>
```

A listagem de candidatos

- Use `th:each` numa `<div>` (ou outro elemento) pra iterar a lista de candidatos vinda do Model:
 - `<div class="card" th:each="cand : {candidatos}">`
- Pra foto, monte a URL concatenando o caminho da pasta com o nome do arquivo (o Candidato já deve ter um método pronto pra isso, ex.: `getNomeArquivoFoto()`):
 - ``
- Use `th:text` para exibir os campos do candidato (nome de urna, número, cargo, partido, etc.).
- Esconda com `th:if` os campos que às vezes vêm vazios ou sem valor útil (situação, idade, ocupação, escolaridade):

```
<p class="situacao" th:if="{cand.situacaoCandidatura != null and
!cand.situacaoCandidatura.isBlank()}"
  th:text="{cand.situacaoCandidatura}">Situação</p>

<p class="extra" th:if="{cand.idade > 0}">
  <span th:text="{cand.idade + ' anos'}">idade</span>
</p>

<p class="extra" th:if="{cand.ocupacao != null and !cand.ocupacao.isBlank()}"
  th:text="{cand.ocupacao}">Ocupação</p>

<p class="extra" th:if="{cand.grauInstrucao != null and !cand.grauInstrucao.isBlank()}"
  th:text="{cand.grauInstrucao}">Escolaridade</p>
```

- Contagem de resultados e lista vazia:

```
<p class="resultado-info">
  <span th:text="${totalEncontrado}">0</span> candidato(s) encontrado(s).
</p>

<p th:if="${totalEncontrado == 0}" class="sem-resultado">
  Nenhum candidato encontrado com esses filtros.
</p>
```

- Os botões do formulário:
 - `<button type="submit">Filtrar</button>`
 - `<a th:href="@{/}" class="limpar">Limpar</a>`

[5,0 pts] Questão 3) Construa o arquivo `static/css/style.css`

Dicas:

- Use CSS Grid pra montar a grade de cards de forma responsiva, sem precisar definir manualmente quantas colunas cabem em cada tamanho de tela:
 - `.grade-candidatos {`
 `display: grid;`
 `grid-template-columns: repeat(auto-fill, minmax(150px, 1fr));`
 `gap: 12px;`
 `}`
- Padronize a proporção da foto com aspect-ratio, evitando fotos esticadas ou cortadas de forma estranha:
 - `.foto-candidato {`
 `width: 100%;`
 `aspect-ratio: 161 / 225;`
 `object-fit: cover;`
 `}`

Observações:

- Não se esqueça de copiar o arquivo `data/candidatos/consulta_cand_2026_MG.csv` para a pasta `resources` do projeto.
 - Copie também a pasta `images/candidatos` para a pasta `static` do projeto.
 - Leia o arquivo `README.md` antes de começar a prova.
-